Delta Lake Optimisation

Partition & Clustering Best Practices

Bas Land

2025-05-08

Kimura
data intelligence

Introduction

Bas Land

- BI consultant since 2013
- Co-founder of Kimura
  - Kimura Data Framework for Fabric
  - Fabric & Power BI Consultancy

- Personal
  - Husband to Anouk
  - Dad to Oliver
  - Dog dad to Chester

- Sports
  - Purple belt Brazilian jiu-jitsu
  - Weight lifting
  - Running

Contact

- Follow me on **LinkedIn**
- Check **ThatFabricGuy.com**
- E-mail **bas.land@kimura.nl**

ThatFabricGuy

LinkedIn

Slides and code samples

- The slides and code samples for this session will be posted to my public Github repo:
- https://github.com/basland/presentations/

- Also check my blog for content like this:
- https://thatfabricguy.com

Performance teaser…

Optimisation	Single Date	Date Range
None	67.47 sec	64.63 sec
Partition by date	3.48 sec	3.94 sec
Cluster by date	7.88 sec	3.48 sec

Why this talk?

- Delta Tables power your Fabric data platform
- Unoptimised Delta = sluggish, expensive, frustrating!
- Modern data engineers must think **physical**
- Delta Lake in Fabric gives us powerful tools & complexity…

What you'll learn today

 Partitioning: get query pruning for free

 Liquid Clustering: continuous optimization on write

 Live Benchmarks: partitioned vs clustered vs unoptimized

 File Compaction & VACUUM: managing the small files beast

 Best Practices & Gotchas

Delta Lake Anatomy

& optimisation principles

About Parquet files

- “Just” text files for structured data

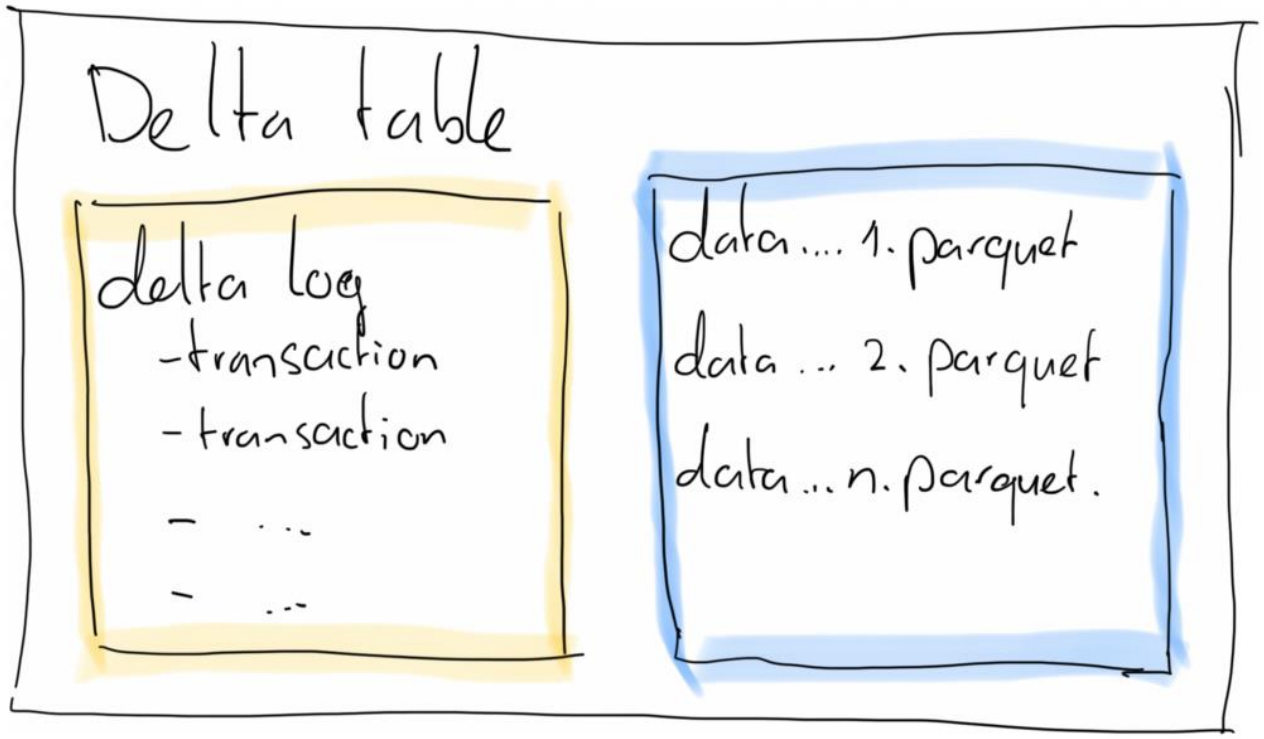
- Columnar storage!
  - Ideal for analytical queries
  - Scanning just a few columns

- Compression by default
  - Less I/O
  - Lower storage costs

- Foundation of Delta Lake tables

What is a Delta Table?

- Delta = Parquet + Transaction log + ACID
- >=1 parquet files
- + transaction log
- + ACID transactions
  - Merge, update, delete
  - (atomic, consistent, isolated, durable)



How queries work

- Delta table = files to be queried as tables

- Query execution:
  1. SQL/Spark query submitted to cluster
  2. Delta log is read for latest snapshot
  3. Min/max stats used to prune (select) files
  4. Relevant parquet files are scanned (only relevant columns)
  5. Result set is prepared and presented (to client, or to data frame, or…)

What are we optimising for?

Goal	Why it matters
Fewer files scanned	Less I/O = faster queries
Smaller file sizes	Faster read (better caching)
Better file layout	Enables skipping on multiple columns
Lower shuffle volume	Efficient joins and aggregations

- Layout = performance
- Query engines don’t index delta tables, they skip files
- Bad layout = full scans, wasted memory, **slow** queries, **burning credit card!**

Fabric-specific enhancements

- Fabric gives us some freebies!

- **V-Order**: VertiPaq-friendly (for DAX) parquet layout

- **DirectLake**: Power BI queries delta tables without import (layout matters even more with **many many many** read queries)

- V-Order is enabled by default.
  Layout optimisation is still **your job**!

Optimisation toolbox

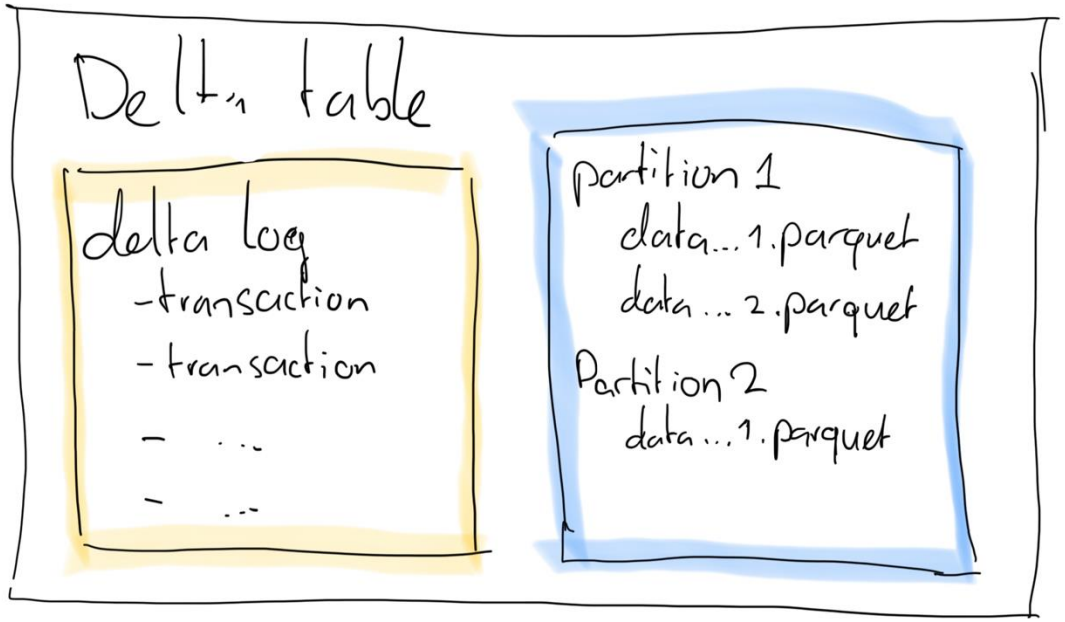
- Four tools for faster Delta Tables

1. Partitioning
2. Liquid Clustering
3. Compacting files (OPTIMIZE)
4. Removing obsolete files (VACUUM)

All about Partitioning

Partitioning – the classic optimisation

- Physically split data in folders (force multiple parquet files)
- Enables **partition pruning**: only read relevant files
- Perfect for **very large tables**
- Partition on **columns used in filters**



Choosing the right column

- Partitioning physically splits the table based on column value
- Use low cardinality columns (region, date, etc)
- Use columns often used in filters

Pitfalls of over-partitioning

- Too many partitions = too many small files
- This results in overhead (reading many files)
- Queries without partition in filter fetch **ALL files**
- Unnecessary for small and medium tables

When is partitioning great?

- For very large tables (100M+ rows or 100GB+ data)
- Consistent query patterns (always filter by X)
- Combine with clustering for multi-column pruning

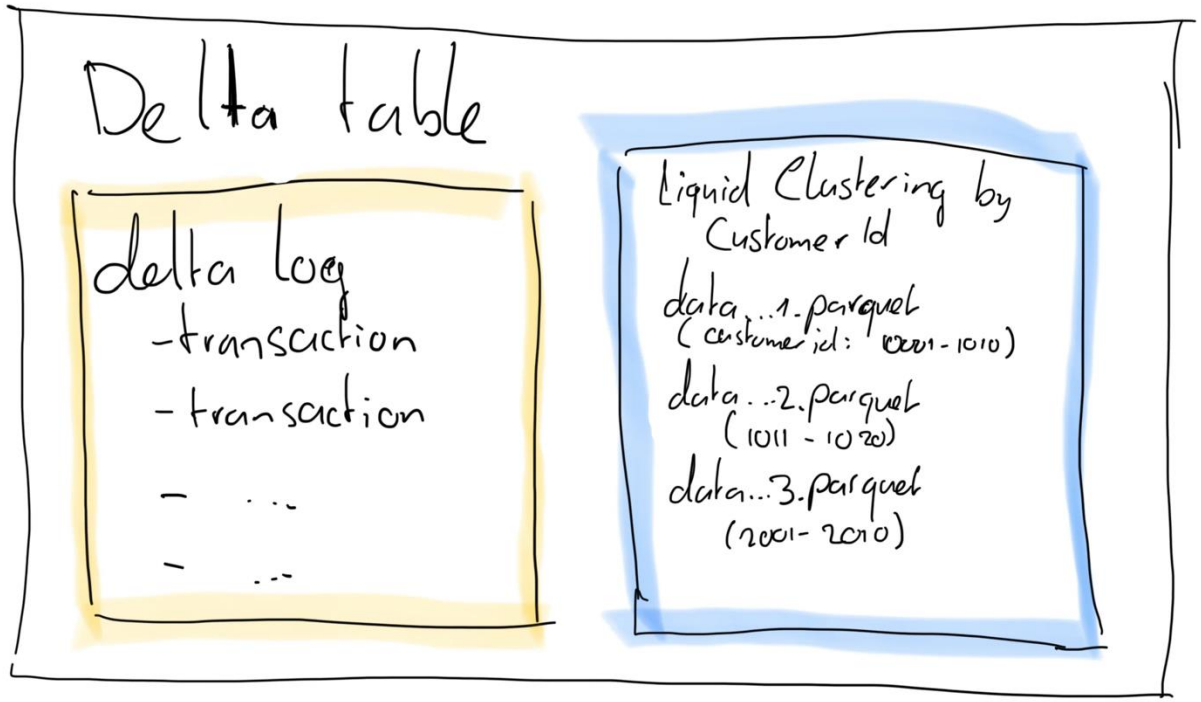
Introducing Liquid Clustering

The limits of partitioning

- Works only for fixed set of columns
- Bad in high-cardinality columns
- Queries without partition filter still scan all files
- Requires fixed schema, difficult to maintain and change later

What is Liquid Clustering?

- Organises data by columns without physical file splits
- Applied incrementally during write, no maintenance job needed
- Rows with similar values end up in the same files
- Helps data skipping on read for filtered queries



Clustering vs Z-Order

Feature	Z-Ordering	Liquid Clustering
How to apply?	Batch (run OPTIMIZE command)	Incremental on write
Compute cost	High (sort + write)	Low to moderate
Maintenance implication	Yes, periodically (e.g. weekly)	No, clustering is automatic
Flexibility	High, but manual	High, automatic

When to use Liquid Clustering

- High cardinality filter columns
- Medium to large tables (10-100GB, 10-100M rows)
- Evolving query patters
- Works well standalone, or combined with partitioning

Live benchmarks

Unoptimised vs optimised tables


Demo time

Wrapping up

File maintenance

- Use Spark commands to maintain your delta files

- OPTIMIZE
  - Merges smaller files into larger files
  - Improves scan speed and reduces read overhead
  - Schedule after frequent inserts/updates, or weekly

- VACUUM
  - Deletes obsolete files not in the Delta log
  - Default retention = 7 days
  - VACUUM frees up storage
  - Run after OPTIMIZE or heavy deletes

- Target 128-1024 MB per file

Conclusion

- Performance in Fabric delta lake is about file layout more than just compute power
- Partitioning can help very large tables, if query patterns are predictable
- Clustering improves file skipping for in a more flexible way and for smaller tables
- Partitioning and clustering can be used together as well when needed

Recommendations

- Use partitioning when:
  - Filters are a fixed, low cardinality column (region, date, …)
  - Table is very large (100M+ rows)
  - Query patterns are predictable or fixed
- Use liquid clustering when:
  - Filters and query patterns evolve over time
  - Tables are larger (10M-100M rows order of magnitude)
- Below 10M rows, don’t bother ☺
- Use EXPLAIN and inputFiles() to verify skipping behaviour
- Use regular OPTIMZE commands (weekly maintenance job) to control files

Q&A

- Any questions? Please ask!